

Explicit MPC Exploratory Report

September 6, 2026

Purpose

This report summarizes the explicit-MPC implementation developed as an exploratory reconstruction of the Eco-MPC controller. It is not a claim that the explicit controller is equivalent to the compiled `libpcc_so.so` model. The original Eco-MPC optimization source, exact weights, and reference construction were not available; therefore, the explicit controller uses a paper-aligned reconstructed QP.

Configuration

Item	Unconnected	Connected
Prediction horizon	32 stages	32 stages
Prediction interval	0.1 s	0.5 s
QP state model	[s, v, a]	[s, v, a]
Parameter dimension	6	38
Preview information	Constant-acceleration PV prediction	32 PV position samples
Region sampling	400 samples	400 samples
Runtime fallback	OSQP if no region matches	OSQP if no region matches
Safety filter	CBF enabled	CBF enabled

The controller generates affine laws of the form

$$u = K_{\text{region}} @ \text{theta} + k_{\text{region}}$$

and verifies the associated KKT inequalities before using a region. The QP contains quadratic gap tracking, acceleration and command penalties, velocity, acceleration-versus-speed, minimum-gap, and slack constraints.

Offline explicit-law validation

The affine laws were checked against OSQP solutions for randomized feasible states.

Case	Discovered regions	Region-covered tests	OSQP fallback tests	Maximum
Unconnected	1	82	0 in covered subset	1.8×10^4
Connected	99	64	27	2.5×10^4

These results show that region evaluation is numerically consistent with the reconstructed QP when a valid region is available. They do not establish agreement with the original compiled Eco-MPC.

SUMO observations

A completed two-vehicle unconnected SUMO run reported:

Statistic	Result
Average controller runtime	1.189 ms
Minimum controller runtime	0.808 ms
Maximum controller runtime	39.945 ms
Runtime standard deviation	1.336 ms
Explicit-region coverage	99.79%
OSQP fallbacks	16
CBF overrides	3,938

The available connected runtime records are approximately 0.59–0.62 ms per recorded controller evaluation, but a completed connected diagnostic run was not retained in the current workspace. These values should therefore be treated as exploratory timing observations rather than final benchmark statistics.

The unconnected trajectory comparison shows the central issue: the original MPC follows the preceding vehicle’s acceleration and deceleration pattern, whereas the reconstructed explicit controller produces a slow monotonic speed rise and responds differently near the end of the cycle. The neural-network controller follows the original MPC much more closely than the explicit reconstruction.

Conclusion

The explicit implementation is numerically valid for its reconstructed QP, but it is not a valid replacement or apples-to-apples benchmark for the original Eco-MPC. The large trajectory difference indicates that the unknown original objective, constraints, prediction conventions, or reference construction materially affect the result. The original compiled MPC should remain the authoritative MPC baseline.

Suggested paper wording

An explicit-MPC implementation was also investigated as an additional benchmark. Because the original Eco-MPC optimization source and exact parameterization were unavailable, the explicit controller had to be reconstructed

from the published model description. Although its affine control laws were numerically consistent with the reconstructed QP, its closed-loop trajectory differed substantially from that of the original compiled Eco-MPC. It was therefore not used as a quantitative benchmark, since such a comparison would not be apples-to-apples. The original MPC was retained as the authoritative baseline, and the neural-network controller was evaluated as the practical approximation.

Reproduction commands

```
python3 scripts/run_sim.py --no_gui --num_sv 5 --print_level info Hwy ExplicitUncon  
python3 scripts/run_sim.py --no_gui --num_sv 5 --print_level info Hwy ExplicitConn  
python3 scripts/run_sim.py --no_gui --num_sv 5 --print_level info Hwy MPC
```

The runtime summaries are stored in `Runtime_Explicit_Unconnected.csv` and `Runtime_Explicit`